SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan

Student Reg. No: 192424061
Name: K.Sandhya

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	**5**
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	**5**
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	**5**
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	**5**
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	**5**

6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	**5**
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

I _K.Sandhya 192424061__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of	25%	Applies advanced and relevant	Applies appropriate concepts	Limited application of concepts	Incorrect or no

Engineering Concepts		concepts effectively to solve the problem	with minor errors		application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. File Organization Strategy for 10 Million Product Records

An e-commerce company storing 10 million product records requires a file organization that supports fast searching, insertion, deletion, and range queries. A B+ Tree-based indexed file organization is recommended.

In a B+ Tree, internal nodes contain search keys and pointers, while actual records or record pointers are stored at the leaf level. The leaf nodes are linked, which makes range searches efficient.

For example, customers may search for:

- Product ID = 105025
- Products in a particular price range
- Products belonging to a particular category
- Products within a particular rating range

Cost Analysis :

File Organization	Search	Insertion	Range Query
Heap	O(N)	O(1)	O(N)
Sorted	O(log N)	O(N)	Good
B+ Tree	O(log N)	O(log N)	Excellent

For 10 million records, a heap file may require millions of records to be checked during an unsuccessful search. A B+ Tree reduces the number of disk accesses significantly.

Conclusion:

A B+ Tree indexed organization is the most suitable because it provides efficient searching, insertion, deletion, and range queries while scaling well to 10 million records.

2. Indexing Strategy for Banking Transactions

A banking system frequently retrieves transactions using Account Number and Date Range**.** Therefore, an appropriate indexing strategy should support both exact and range searches.

A **composite B+ Tree index** on:

(Account_No, Transaction_Date)

is recommended.

Example:

CREATE INDEX idx_account_date

ON Transactions(Account_No, Transaction_Date);

For example, the following query can be efficiently processed:

```
SELECT *
FROM Transactions
WHERE Account_No = 101
AND Transaction_Date BETWEEN '2026-01-01' AND '2026-01-31';
```

The index first locates the required account number and then searches the required date range.

Advantages :

1. Account number lookup is fast.
2. Date-range searches are efficient.
3. B+ Tree supports ordered traversal.
4. Linked leaf nodes reduce the cost of retrieving consecutive transactions.
5. It reduces disk I/O compared with a full table scan.

Conclusion:

A composite B+ Tree index on (Account_No, Transaction_Date) is the best choice for fast transaction retrieval.

3. Multi-Level Indexing for Healthcare Records

A healthcare system stores a large number of patient records. Records must be accessed using both PatientID and Doctor Name**.** Therefore, both primary and secondary indexes are required.

Proposed Structure :

Primary B+ Tree Index:

PatientID → Patient Record

Secondary B+ Tree Index:

Doctor Name → PatientID

For example:

PatientID
↓
1050 → Patient Record
1051 → Patient Record
1052 → Patient Record

The doctor index may contain:

Dr. Kumar → 1050, 1055, 1080
Dr. Priya → 1051, 1060

A multi-level index is useful because the index itself can be divided into multiple levels. The top-level index points to lower-level index blocks, which finally point to the required records.

Advantages

- Fast PatientID searching.
- Fast doctor-based searching.
- Reduces disk access.

- Supports large healthcare databases.
- B+ Tree provides efficient range and sequential access.

Conclusion:

Use a primary B+ Tree index on PatientID and a secondary B+ Tree index on Doctor Name**.** This provides efficient access through both search fields.

4. B-Tree vs B+ Tree for Logistics Company

A logistics company frequently inserts and deletes shipment records. It may also need to retrieve shipments based on shipment ID, location, delivery date, or status.

Both B-Tree and B+ Tree support dynamic insertion and deletion, but B+ Tree is more suitable for database systems**.**

B-Tree

In a B-Tree, keys and records may be stored in both internal and leaf nodes. Searching can finish at an internal node.

B+ Tree

In a B+ Tree, internal nodes contain only keys and pointers, while actual records are stored at the leaf level. The leaf nodes are connected through pointers.

Feature	B-Tree	B+ Tree
Search	Fast	**Fast**
Insert	Efficient	**Efficient**
Delete	Efficient	**Efficient**
Range queries	Good	**Excellent**
Sequential access	Good	**Excellent**
Database suitability	Good	**Very high**

For example, finding all shipments scheduled between two dates is much easier with the linked leaf nodes of a B+ Tree.

Conclusion:

B+ Tree is recommended because it handles dynamic insertions and deletions efficiently and provides excellent range and sequential access.

5. Hashing Scheme for HR System with 5000 Employees

An HR system contains 5000 employee records. Hashing can provide fast access to employee records using Employee_ID.

A simple hash function can be:

h(Employee_ID)=Employee_ID MOD 100

This creates 100 initial buckets.

For example:

Employee ID = 1250

1250 MOD 100 = 50

Therefore, the record is stored in bucket 50.

Static Hashing

In static hashing, the number of buckets is fixed.

Advantages:

- Simple implementation.
- Fast exact search.
- Average search time is O(1).
- Easy to maintain when the database size is stable.

Disadvantage:

If employee records increase significantly, overflow buckets may be required.

Extendible Hashing

Extendible hashing dynamically increases the directory and splits buckets when they become full.

Feature	Static Hashing	Extendible Hashing
Number of buckets	Fixed	Dynamic
Search	O(1) average	O(1) average
Growth	Difficult	**Easy**
Overflow handling	Overflow chains	Bucket splitting
Complexity	Simple	Higher

Conclusion:

If the HR database will remain close to 5000 employees, static hashing is sufficient and economical. If the organization is expected to grow, extendible hashing is better**.**

6. Combined Index for Airline Reservation System

An airline reservation system requires two different types of queries:

1. Point query using Flight Number.
2. Range query using Flight Date.

A single index may not be ideal for both operations. Therefore, two B+ Tree indexes can be created.

Flight_Number → B+ Tree Index
Flight_Date → B+ Tree Index

Example:

```
CREATE INDEX idx_flight_number
ON Reservations(Flight_Number);

CREATE INDEX idx_flight_date
ON Reservations(Flight_Date);
```

For an exact query:

```
SELECT *
FROM Reservations
WHERE Flight_Number = 'AI202';
```

the Flight Number index provides fast retrieval.

For a range query:

```
SELECT *
FROM Reservations
WHERE Flight_Date
BETWEEN '2026-08-01' AND '2026-08-15';
```

the Date B+ Tree efficiently searches the required range.

Advantages :

- Fast flight-number lookup.
- Efficient date-range retrieval.
- Reduced disk I/O.
- B+ Tree supports ordered traversal.
- Suitable for large reservation databases.

Conclusion:

Use separate B+ Tree indexes on Flight_Number and Flight_Date to efficiently support both point and range queries.

7. University Student Database File Organization

A university may store thousands of student records. The system needs fast queries based on Department and CGPA range**.**

A suitable solution is to use a sorted file organization with secondary B+ Tree indexes.

The primary file can be sorted using Student_ID.

Secondary indexes:

Department → Student Records
CGPA → Student Records

For example:

Department Index

CSE → S001, S005, S009
ECE → S002, S006, S010
IT → S003, S007

The CGPA index can be:

CGPA

7.5 → Student records
8.0 → Student records
8.5 → Student records
9.0 → Student records

A query such as:

```
SELECT *
FROM Student
WHERE Department = 'CSE';
```

can use the Department index.

Similarly:

```
SELECT *
FROM Student
WHERE CGPA BETWEEN 8.0 AND 9.0;
```

can use the CGPA B+ Tree.

Conclusion:

A sorted file with secondary B+ Tree indexes on Department and CGPA provides fast departmental searches and efficient CGPA range queries.

8. Disk Block Utilization Analysis

Given:

- Number of records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

Number of records that can fit in one block:

504096=81.92

Therefore:

81 records/block

Number of blocks required:

81100000=1234.57

Therefore, approximately:

1235 blocks

Actual space used by 81 records:

81×50=4050 bytes

Unused space:

4096−4050=46 bytes

Block utilization:

40964050×100≈98.88%

Comparison

Organization	Utilization	Search	Insert
Heap	~98.88%	O(N)	**O(1)**
Sorted	~98.88%	**O(log N)**	O(N)
Hashed	Depends on load	**O(1) average**	**O(1) average**

Conclusion:

Approximately 1235 blocks are required, with about 98.88% space utilization**.** Heap is best for fast insertion, sorted files are best for ordered/range searches, and hashing is best for exact-key searches.

9. Composite Indexing for 500 Million Social Media Posts

A social media platform has 500 million posts and needs to search using:

- user_id
- timestamp
- hashtag

Because of the huge data size, proper composite indexes are required.

Recommended Indexes

CREATE INDEX idx_user_time

```
ON Posts(user_id, timestamp);

CREATE INDEX idx_hashtag_time
ON Posts(hashtag, timestamp);
```

The first index is useful for queries such as:

```
SELECT *
FROM Posts
WHERE user_id = 1001
ORDER BY timestamp;
```

The second index is useful for:

```
SELECT *
FROM Posts
WHERE hashtag = '#cricket'
AND timestamp BETWEEN '2026-08-01' AND '2026-08-24';
```

Why Composite Indexes?

The first attribute is used to identify the group of records, while the second attribute helps efficiently search or order records within that group.

For 500 million records, indexing significantly reduces the number of records that need to be examined.

Additional optimization: The data can be **partitioned by time**, such as monthly partitions, to reduce the amount of data scanned for recent posts.

Conclusion: Use **B+ Tree composite indexes (user_id, timestamp) and (hashtag, timestamp)**, along with time-based partitioning for scalability.

---

10. Comparison of Heap, Sorted and Hashed Files

A supermarket billing system handles approximately **1 million transactions per day**. It requires fast insertion of new bills and quick retrieval of previous transactions.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	**O(1)**	O(N)	**O(1) average**
Search	O(N)	O(log N)	**O(1) average**
Delete	O(N)	O(N)	**O(1) average**
Range Query	O(N)	**Excellent**	Poor
Best for	Frequent inserts	Range searches	Exact searches

Heap File

New transactions can simply be added at the end of the file. Therefore, insertion is very fast. However, searching for a particular transaction may require scanning the entire file.

Sorted File

Records are maintained in sorted order. Binary search can provide fast searching, and range queries are very efficient. However, inserting a new transaction may require moving existing records, making insertion expensive.

Hashed File

A hash function maps a transaction ID to a storage bucket. This provides very fast average access for exact searches.

For example:

Hash(Transaction_ID) → Bucket → Transaction

However, hashing is not suitable for range queries such as:

Transactions between ₹10,000 and ₹20,000

Recommendation

For a supermarket billing system, hashed file organization is best for exact Transaction_ID searches and fast daily insertions**.** If the system also performs frequent date-based reports, a B+ Tree secondary index on Transaction_Date should be added.

conclusion:

A combination of Hashing + B+ Tree secondary indexing provides the best overall performance for a high-volume supermarket billing system.